

Math 151A Homework 5

Tiya Chokhani

Q1.

Let the knots be $x_0 = 0$, $x_1 = 1$, $x_2 = 2$ with $y_i = f(x_i) = i$. For a natural cubic spline, the second derivatives at the nodes $M_i = s''(x_i)$ satisfy

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right), \quad i = 1,$$

with $M_0 = 0$, $M_2 = 0$, and $h_0 = h_1 = 1$. Thus

$$M_0 + 4M_1 + M_2 = 6((2 - 1) - (1 - 0)) = 0 \Rightarrow M_1 = 0.$$

The cubic on each subinterval $[x_i, x_{i+1}]$ is

$$s(x) = \frac{M_i(x_{i+1} - x)^3}{6h_i} + \frac{M_{i+1}(x - x_i)^3}{6h_i} + \left(y_i - \frac{M_i h_i^2}{6}\right) \frac{x_{i+1} - x}{h_i} + \left(y_{i+1} - \frac{M_{i+1} h_i^2}{6}\right) \frac{x - x_i}{h_i}.$$

With $M_0 = M_1 = M_2 = 0$ and $h_i = 1$, this reduces to linear interpolation on both subintervals,

$s(x) = x \quad \text{for } 0 \leq x \leq 2.$

Q2.

Let $\hat{s}$ be the natural cubic spline interpolant of data (x_i, y_i) with $x_0 < \dots < x_n$, so $\hat{s} \in C^2[x_0, x_n]$, piecewise cubic on $[x_i, x_{i+1}]$, and $\hat{s}''(x_0) = \hat{s}''(x_n) = 0$. For any $g \in C^2[x_0, x_n]$ with the same nodal values $g(x_i) = y_i$, define $e = g - \hat{s}$. Then $e(x_i) = 0$ for all i . Compute

$$\int_{x_0}^{x_n} (g'')^2 = \int_{x_0}^{x_n} (\hat{s}'' + e'')^2 = \int_{x_0}^{x_n} (\hat{s}'')^2 + 2 \int_{x_0}^{x_n} \hat{s}'' e'' + \int_{x_0}^{x_n} (e'')^2.$$

We show the cross term is zero. On each $[x_i, x_{i+1}]$,

$$\int_{x_i}^{x_{i+1}} \hat{s}'' e'' = [\hat{s}'' e']_{x_i}^{x_{i+1}} - \int_{x_i}^{x_{i+1}} \hat{s}''' e' dx = [\hat{s}'' e']_{x_i}^{x_{i+1}} - [\hat{s}''' e]_{x_i}^{x_{i+1}} + \int_{x_i}^{x_{i+1}} \hat{s}^{(4)} e dx.$$

Since $\hat{s}$ is cubic on each subinterval, $\hat{s}^{(4)} \equiv 0$. Also $e(x_i) = e(x_{i+1}) = 0$, so the middle boundary term vanishes. Summing over subintervals telescopes the first boundary term, giving

$$\int_{x_0}^{x_n} \hat{s}'' e'' = \hat{s}''(x_n) e'(x_n) - \hat{s}''(x_0) e'(x_0) = 0$$

by the natural conditions $\hat{s}''(x_0) = \hat{s}''(x_n) = 0$. Hence

$$\int (g'')^2 = \int (\hat{s}'')^2 + \int (e'')^2 \geq \int (\hat{s}'')^2,$$

with equality only if $e'' \equiv 0$ and therefore $g = \hat{s}$ (since e vanishes at all nodes). Thus

$$\boxed{\int_{x_0}^{x_n} (\hat{s}''(x))^2 dx \leq \int_{x_0}^{x_n} (g''(x))^2 dx.}$$

Q3.

We compare cubic splines with **natural** and **clamped** boundary conditions on $[0, 1]$ for

$$(i) f(x) = e^x, \quad (ii) f(x) = \sin x, \quad (iii) f(x) = x\sqrt{x} = x^{3/2}.$$

For clamped splines we use exact endpoint slopes: $f'(x) = e^x$, $f'(x) = \cos x$, and $f'(x) = \frac{3}{2}\sqrt{x}$ with $f'(0) = 0$.

We measure the max error on a dense grid and estimate rates by halving the mesh size h . Below is a simple Python script (uses **numpy** and **scipy**):

```
import numpy as np
from scipy.interpolate import CubicSpline

def funcs():
    return [
        ("exp", lambda x: np.exp(x), lambda x: np.exp(x)),
        ("sin", lambda x: np.sin(x), lambda x: np.cos(x)),
        ("x32", lambda x: x*np.sqrt(x), lambda x: 1.5*np.sqrt(x)),
    ]

def max_err(f, s, grid):
    return np.max(np.abs(s(grid) - f(grid)))

xs_dense = np.linspace(0,1,20001)

for name, f, fp in funcs():
    print("\n==", name, "==")
    N = 8
    for k in range(5):
        x = np.linspace(0,1,N+1)
        y = f(x)
        s_nat = CubicSpline(x, y, bc_type='natural')
        s_cla = CubicSpline(x, y, bc_type=((1, fp(0.0)), (1, fp(1.0))))
```

```

e_nat = max_err(f, s_nat, xs_dense)
e_cla = max_err(f, s_cla, xs_dense)

print(f"N={N:4d}    |E|_nat={e_nat:.3e}    |E|_clamped={e_cla:.3e}")
N *= 2

```

Expected observations.

- For smooth $f \in C^4[0, 1]$ such as e^x and $\sin x$, both splines show about $O(h^4)$ max error. Clamped usually improves accuracy near $x = 0$ and $x = 1$ because the endpoint slope is enforced.
- For $x^{3/2}$, $f''(x) = \frac{3}{4}x^{-1/2}$ is unbounded at 0. Convergence degrades below fourth order. Natural splines force $s''(0) = 0$ which mismatches the true curvature and often increases endpoint error. Clamped splines use $f'(0) = 0$ and can reduce the visible kink, but the singular second derivative still limits the rate.

Q4.

I reviewed the two endpoint choices from Section 3.5: *natural* splines with $S''(x_0) = S''(x_n) = 0$ and *clamped* splines with $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$. In the standard construction the cubic on $[x_j, x_{j+1}]$ is

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3,$$

and the coefficients are found from a tridiagonal system for $\{c_j\}$ that enforces continuity of S , S' , and S'' at the interior nodes. The boundary choice changes only the two end equations:

$$\text{natural: } c_0 = 0, c_n = 0, \quad \text{clamped: } \begin{cases} b_0 = \frac{a_1 - a_0}{h_0} - \frac{h_0}{3}(2c_0 + c_1) = f'(x_0), \\ b_{n-1} = f'(x_n) = \frac{a_n - a_{n-1}}{h_{n-1}} + \frac{h_{n-1}}{3}(2c_{n-1} + c_n), \end{cases}$$

with $h_j = x_{j+1} - x_j$ and $a_j = f(x_j)$. After solving for $\{c_j\}$ we recover

$$b_j = \frac{a_{j+1} - a_j}{h_j} - \frac{h_j}{3}(2c_j + c_{j+1}), \quad d_j = \frac{c_{j+1} - c_j}{3h_j}.$$

Takeaways.

- Natural splines are a good default when endpoint slopes are unknown. They minimize the bending energy $\int (S'')^2$ among all twice differentiable interpolants with the same nodes, so the shape is smooth and unbiased at the ends.

- If $f'(x_0)$ and $f'(x_n)$ are known and not near zero, clamped splines usually reduce boundary error and often improve integral estimates of f over the interval, since S' matches f' at the ends.
- When the true endpoint slopes are wrong or very noisy, clamped conditions can hurt accuracy. In that case the natural choice is safer.

I chose this topic because it is a small extension of what we do in class that is easy to implement, and it explains a practical decision I need to make in code: whether to use natural or clamped endpoints for better accuracy near the boundaries.